Lab Work — Linear Models for Regression

The NO2 dataset provides 500 observations of air pollution largely associated with road traffic. In this lab, your task is to **predict the NO_2 concentration** using information about traffic intensity and local atmospheric conditions. Example predictors include vehicles per hour, temperature, wind-related measurements, and indicators for the time of day.

```
# Auto-setup when running on Google Colab
if 'google.colab' in str(get_ipython()):
    !pip install openml

# General imports
%matplotlib inline
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import openml as oml
from matplotlib import cm

# Hide convergence warning for now
import warnings
from sklearn.exceptions import ConvergenceWarning
with warnings.catch_warnings():
    warnings.filterwarnings("ignore", category=ConvergenceWarning)
```

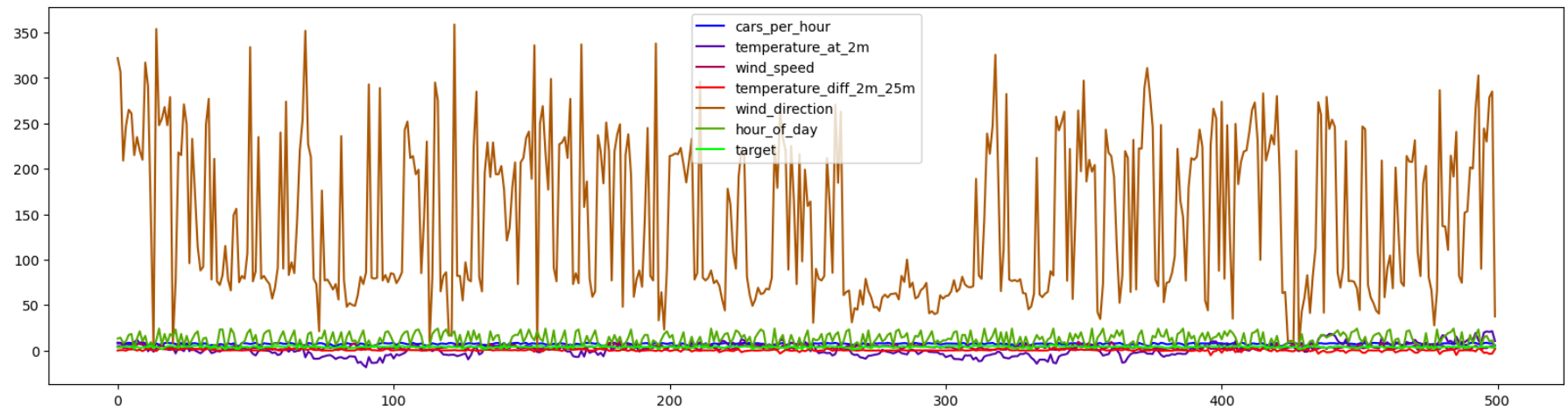
```
# Download NO2 data. Takes a while the first time.
no2 = oml.datasets.get_dataset(547)
X, y, _, _ = no2.get_data(target=no2.default_target_attribute);
attribute_names = list(X)
```

Data

We can use **pandas** to get a quick sense of the dataset. If pandas is new to you, pause here and read the code carefully—it's worth understanding.

To keep things focused, we will drop the `day` feature and concentrate on the *non-calendar* drivers. This activity is **not** a forecasting task; forecasting would require a different setup (for example, time-aware train/test splits). Also, there is no obvious long-term drift in the series, although temperature clearly shows repeating (periodic) patterns.

```
df = pd.DataFrame(X, columns=attribute_names).join(pd.DataFrame(list(y),columns=['target']))
df = df.sort_values(['day','hour_of_day']).drop('day',axis=1)
df.plot(use_index=False,figsize=(20,5),cmap=plt.colormaps['brg']);
X = X.drop('day',axis=1)
```

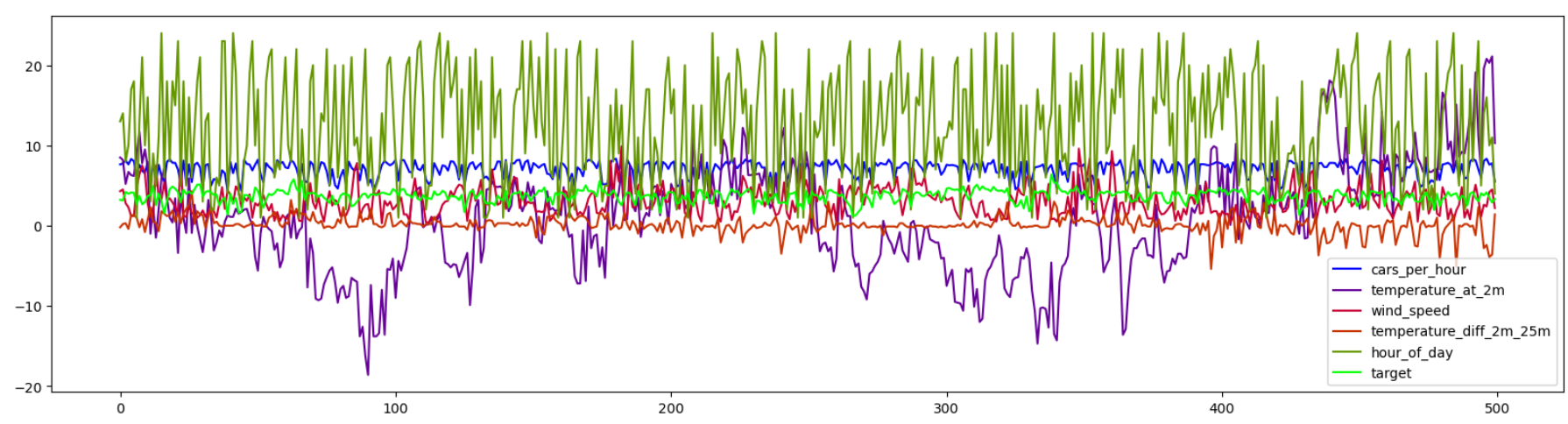



```
df.head()
```

	cars_per_hour	temperature_at_2m	wind_speed	temperature_diff_2m_25m	wind_direction	hour_of_day	target
42	7.64300	8.5	4.3	-0.2	322.0	13	3.22287
20	7.75061	8.2	4.5	0.2	307.0	14	3.15274
255	8.12415	5.2	2.8	0.3	209.0	8	4.19570
488	7.64108	6.7	2.3	-0.4	247.0	10	3.98155
94	8.31630	6.3	1.2	1.3	265.0	17	4.14155

When we plot the measurements in their recorded order, you'll notice that `wind_direction` (an angle in degrees) lives on a very different numeric scale than the other variables. To make patterns easier to see, let's zoom in on the remaining features.

```
df.drop('wind_direction',axis=1).plot(use_index=False, figsize=(20,5), cmap=plt.colormaps['brg']);
```



The target (NO_2 concentration) appears to move together with the *traffic count* (cars per hour), which is reasonable since vehicle emissions contribute to NO_2. Other factors—such as temperature and wind—seem to influence the outcome in less direct ways. Next, we'll try to capture these relationships using **linear models**.

Exercise 1: Baseline comparison

Because NO_2 depends on multiple inputs at once, we will learn a model that combines them. Start by comparing several methods: **Linear Regression**, **Ridge**, **Lasso**, **ElasticNet**, and **kNN**. Since some inputs have very different scales, you will also evaluate pipeline versions that include a **scaling** step. For now, keep all hyperparameters at their default values.

Exercise 1.1 - GUMBAN

Write a function that takes a list of models and evaluates each one on the provided data. Your function should return two lists: **training scores** and **test scores** (in the same order as the models). You may import any helpers you need, but you should be able to do this with `cross_validate` and standard Python tools. An example of the expected output is shown below the function.

```
def evaluate_learners(models, X, y):
    """
    Given a list of models [model1, model2, ..., modelN] return two lists:
     - a list with the scores obtained on the training samples for each model,
     - a list with the test scores obtained on the test samples for each model.
     The order of scores should match the order in which the models were originally provided. E.g.:
     [Model1 train score, ..., ModelN train score], [Model1 test score, ..., ModelN test score]
    """
    from sklearn.model_selection import cross_validate
    train_scores = []
    test_scores = []
    for model in models:
        cv_results = cross_validate(model, X, y, cv=5, return_train_score=True, scoring='r2')
        train_scores.append(cv_results['train_score'].tolist())
        test_scores.append(cv_results['test_score'].tolist())

    # Print in example format
    print("train_scores, test_scores = (");
    for i, row in enumerate(train_scores):
        fmt = [round(x, 3) for x in row]
        print("  ", fmt, ",  # Model", i + 1, "train score for each of 5 folds.");
    print(" " * 14 + "..");
    for i, row in enumerate(test_scores):
        fmt = [round(x, 3) for x in row]
        print(" " * 14, fmt, ",  # Model", i + 1, "test score for each of 5 folds.");
    print(" " * 14 + "..");
    print(")");
    return train_scores, test_scores

# # Example output:
# train_scores, test_scores = ([[0.92 , 0.924, 0.916, 0.917, 0.921],  # Model 1 train score for each of 5 folds.
#                               [0.963, 0.962, 0.953, 0.912, 0.934],  # Model 2 train score for each of 5 folds.
#                               ..
#                              [[0.801, 0.811, 0.806, 0.826, 0.804],  # Model 1 test score for each of 5 folds.
#                               [0.766, 0.756, 0.773, 0.756, 0.741],  # Model 2 test score for each of 5 folds.
#                               ..
```

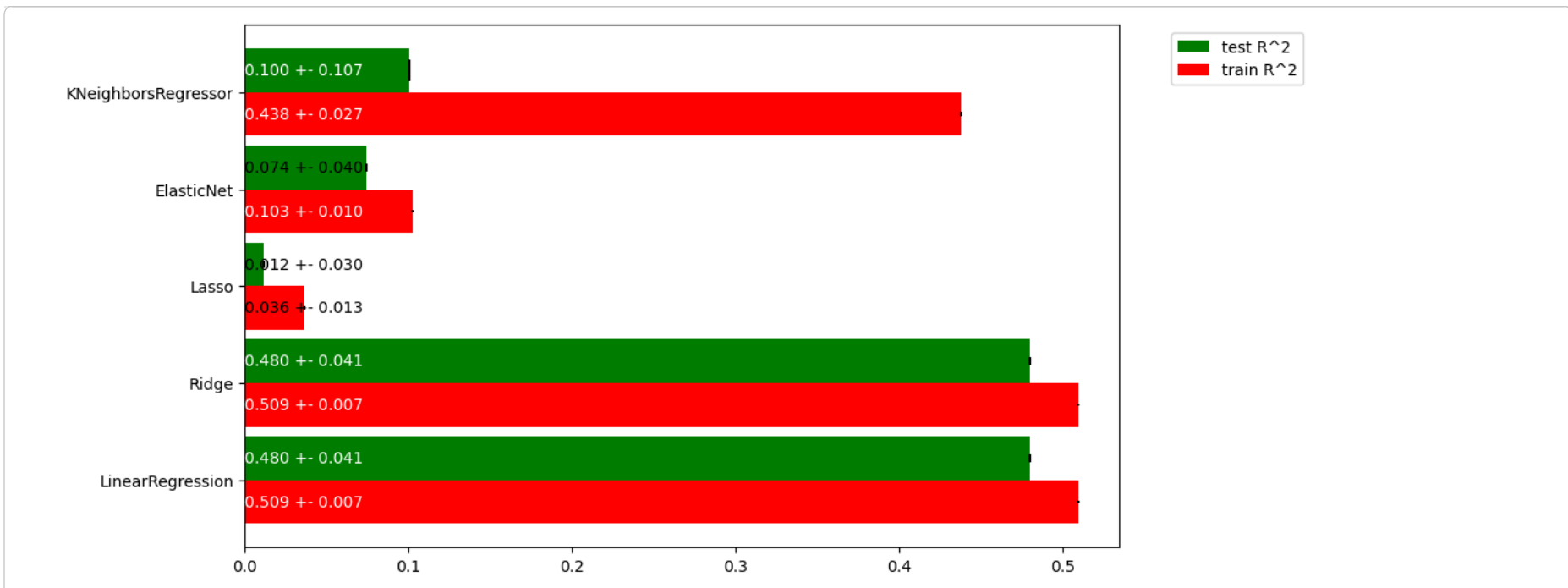
Exercise 1.2 - ABELARDO

Use your evaluation function on: Linear Regression, Ridge, Lasso, ElasticNet, and kNN. Save the results into `train_scores` and `test_scores`, then run the plotting code below to visualize the comparison.

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.neighbors import KNeighborsRegressor

models = [
    LinearRegression(),
    Ridge(),
    Lasso(),
    ElasticNet(),
    KNeighborsRegressor()
]

train_scores, test_scores = evaluate_learners(models, X, y)
```

```
train_scores, test_scores = (
   [0.516, 0.503, 0.519, 0.507, 0.501] ,  # Model 1 train score for each of 5 folds.
   [0.516, 0.503, 0.519, 0.507, 0.501] ,  # Model 2 train score for each of 5 folds.
   [0.032, 0.032, 0.062, 0.029, 0.026] ,  # Model 3 train score for each of 5 folds.
   [0.097, 0.104, 0.12, 0.096, 0.095] ,  # Model 4 train score for each of 5 folds.
   [0.437, 0.458, 0.474, 0.394, 0.427] ,  # Model 5 train score for each of 5 folds.
              ..
               [0.454, 0.512, 0.414, 0.494, 0.526] ,  # Model 1 test score for each of 5 folds.
               [0.454, 0.513, 0.414, 0.494, 0.526] ,  # Model 2 test score for each of 5 folds.
               [0.044, 0.028, -0.035, -0.013, 0.033] ,  # Model 3 test score for each of 5 folds.
               [0.105, 0.093, -0.001, 0.069, 0.105] ,  # Model 4 test score for each of 5 folds.
               [0.058, 0.123, -0.08, 0.239, 0.162] ,  # Model 5 test score for each of 5 folds.
              ..
)
```

```
# Plot a bar chart of the train and test scores of all the classifiers, including the variance as error bars
fig, ax = plt.subplots(figsize=(10,6))
width=0.45

ax.barh(np.arange(len(train_scores)), np.mean(test_scores, axis=1), width,
        yerr= np.std(test_scores, axis=1), color='green', label='test R^2')
ax.barh(np.arange(len(train_scores))-width, np.mean(train_scores, axis=1), width,
        yerr= np.std(train_scores, axis=1), color='red', label='train R^2')
for i, te, tr in zip(np.arange(len(train_scores)),test_scores,train_scores):
    ax.text(0, i, "{:.3f} +- {:.3f}".format(np.mean(te),np.std(te)), color=('white' if np.mean(te)>0.1 else 'black'), va='center')
    ax.text(0, i-width, "{:.3f} +- {:.3f}".format(np.mean(tr),np.std(tr)), color=('white' if np.mean(tr)>0.1 else 'black'), va='center')
labels = [c.__class__.__name__ if not hasattr(c, 'steps') else c.steps[0][0] + "_" + c.steps[1][0] for c in models]
ax.set(yticks=np.arange(len(train_scores))-width/2, yticklabels=labels)
ax.legend(bbox_to_anchor=(1.05, 1), loc=2)

plt.show()
```



Exercise 1.3 - ABELARDO

Explain what the plot is telling you. Which regressor looks strongest? Do any models appear to be overfitting? If yes, what adjustment(s) could reduce overfitting? Also comment on whether the results vary a lot across folds (i.e., the amount of variance).

Strongest Regressor:

Linear Regression and Ridge are the strongest performers. They share the highest test R^2 score (≈ 0.480) and show the best generalization since their train and test scores are nearly identical.

Overfitting Models

KNeighborsRegressor is clearly overfitting. It has a relatively high training score (0.438) but a very poor test score (0.100), indicating it fails to generalize to new data. To reduce this, we could increase the number of neighbors (k) to smooth the model's predictions.

Underfitting Models

Lasso and ElasticNet are underfitting, as both their train and test scores are near zero. This suggests the regularization penalty is too high for this dataset.

Variance Across Folds

Low Variance: Linear Regression and Ridge are very stable, with a low standard deviation (± 0.041 on test).

High Variance: KNeighborsRegressor shows the most significant variance (± 0.107), meaning its performance fluctuates heavily depending on the specific data fold.

Exercise 2: Regularization tuning

Next, tune the main *complexity control* for each method: `alpha` for Ridge and Lasso, and `n_neighbors` for kNN.

Search ranges:

- $\alpha \in [10^{-12}, 10^{12}]$ (use a **log scale**, e.g., 0.01, 0.1, 1, 10, 100)
- `n_neighbors` from **1 to 50** (use a **uniform** grid, e.g., 1, 2, 3, 4, ...)

Exercise 2.1 - GUMBAN

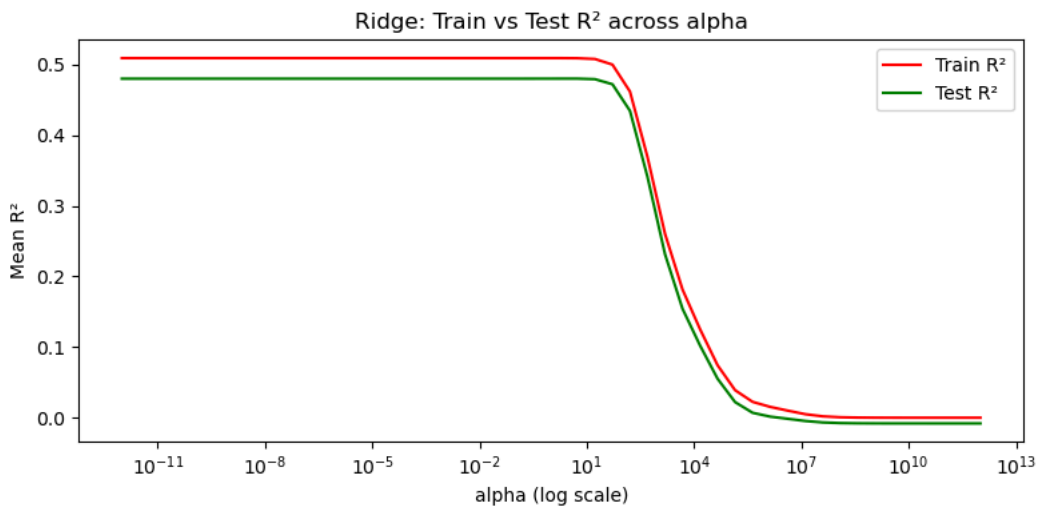
For each regressor, sweep the hyperparameter ranges above and create a line plot showing **mean training score** and **mean test score** versus the hyperparameter value (5-fold CV is fine). You should end up with **three** plots: one for Ridge/Lasso (alpha) and one for kNN (neighbors).

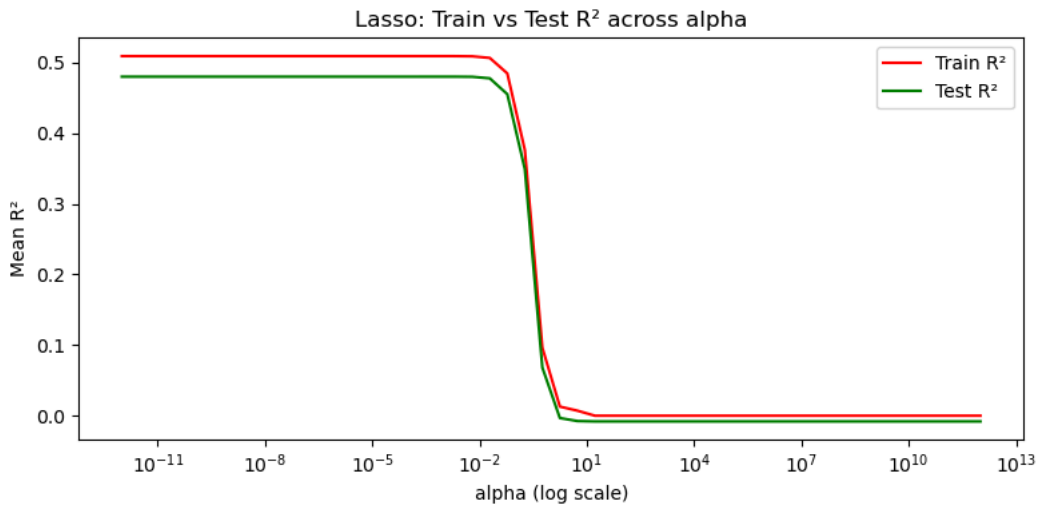
Hints:

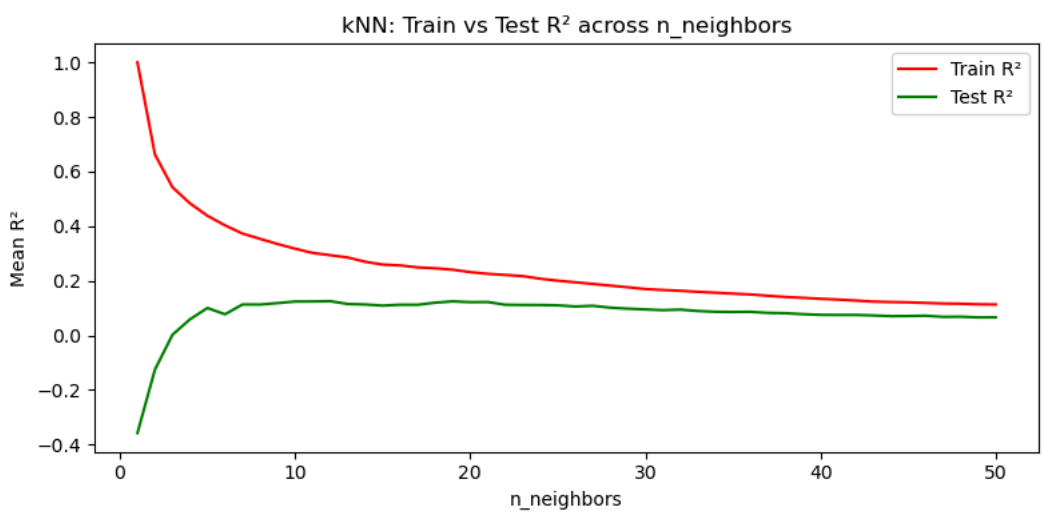
- Be mindful of runtime—too many settings can get slow.
- Useful NumPy helpers: logspace, geomspace, and linspace.
- For plotting, Matplotlib's basic plot function is enough.
- You can loop manually, or you can explore GridSearchCV to automate the search (we will revisit it later).

```
from sklearn.linear_model import Ridge, Lasso
from sklearn.neighbors import KNeighborsRegressor
from sklearn.model_selection import cross_validate

# --- Hyperparameter grids ---
alphas = np.logspace(-12, 12, 50)       # log-spaced alphas
neighbors = np.arange(1, 51)            # 1 to 50 inclusive

def sweep_hyperparameter(ModelClass, param_name, param_values, X, y, cv=5):
    """Sweep a single hyperparameter and return mean train/test R² scores."""
    mean_train, mean_test = [], []
    for val in param_values:
        model = ModelClass(**{param_name: val})
        results = cross_validate(model, X, y, cv=cv,
                                 return_train_score=True, scoring='r2')
        mean_train.append(results['train_score'].mean())
        mean_test.append(results['test_score'].mean())
    return np.array(mean_train), np.array(mean_test)

# Sweep
ridge_train, ridge_test = sweep_hyperparameter(Ridge, 'alpha', alphas, X, y)
lasso_train, lasso_test = sweep_hyperparameter(Lasso, 'alpha', alphas, X, y)
knn_train,   knn_test   = sweep_hyperparameter(KNeighborsRegressor, 'n_neighbors', neighbors, X, y)

# Plot 1: Ridge vs alpha
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(alphas, ridge_train, label='Train R²', color='red')
ax.plot(alphas, ridge_test,  label='Test R²',  color='green')
ax.set_xscale('log')
ax.set_xlabel('alpha (log scale)')
ax.set_ylabel('Mean R²')
ax.set_title('Ridge: Train vs Test R² across alpha')
ax.legend()
plt.tight_layout()
plt.show()
```

```
# Plot 2: Lasso vs alpha
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(alphas, lasso_train, label='Train R²', color='red')
ax.plot(alphas, lasso_test,  label='Test R²',  color='green')
ax.set_xscale('log')
ax.set_xlabel('alpha (log scale)')
ax.set_ylabel('Mean R²')
ax.set_title('Lasso: Train vs Test R² across alpha')
ax.legend()
plt.tight_layout()
plt.show()

# Plot 3: kNN vs n_neighbors
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(neighbors, knn_train, label='Train R²', color='red')
ax.plot(neighbors, knn_test,  label='Test R²',  color='green')
ax.set_xlabel('n_neighbors')
ax.set_ylabel('Mean R²')
ax.set_title('kNN: Train vs Test R² across n_neighbors')
ax.legend()
plt.tight_layout()
plt.show()
```







Exercise 2.2 - BALINGIT

Discuss the curves you obtained. Where do the models underfit? Where do they overfit? How much does performance change when you tweak the regularization setting?

Where do the models underfit?:

- Ridge and Lasso: These models underfit when alpha is high (right side of the plots, roughly $alpha > 10^1$). As alpha increases, the penalty becomes too restrictive, causing both train and test R^2 to drop toward zero.
- kNN: Underfitting occurs as n_neighbors increases (moving toward 50). The model becomes too simple, leading to a gradual decline in both training and test performance.

Where do they overfit?:

- Ridge and Lasso: Overfitting is minimal here, but it occurs at very low alpha values (left side of the plots) where the model behaves like standard Linear Regression.
- kNN: Significant overfitting occurs when n_neighbors is very low (e.g., $n = 1$). You can see the training R^2 is near 1.0 while the test R^2 is negative, indicating the model has memorized the training noise.

Performance change with tuning:

- Ridge/Lasso: Performance is stable across a wide range of small alphas but "crashes" once the penalty exceeds a certain threshold.
- kNN: Increasing $n_neighbors$ drastically improves the test score initially (fixing the overfitting) before it slowly tapers off into underfitting.

Exercise 2.3 - BALINGIT

ElasticNet blends L1 and L2 regularization. The `l1_ratio` parameter controls how much weight goes to L1 (Lasso) versus L2 (Ridge), so ElasticNet has **two** interacting hyperparameters: `l1_ratio` and `alpha`.

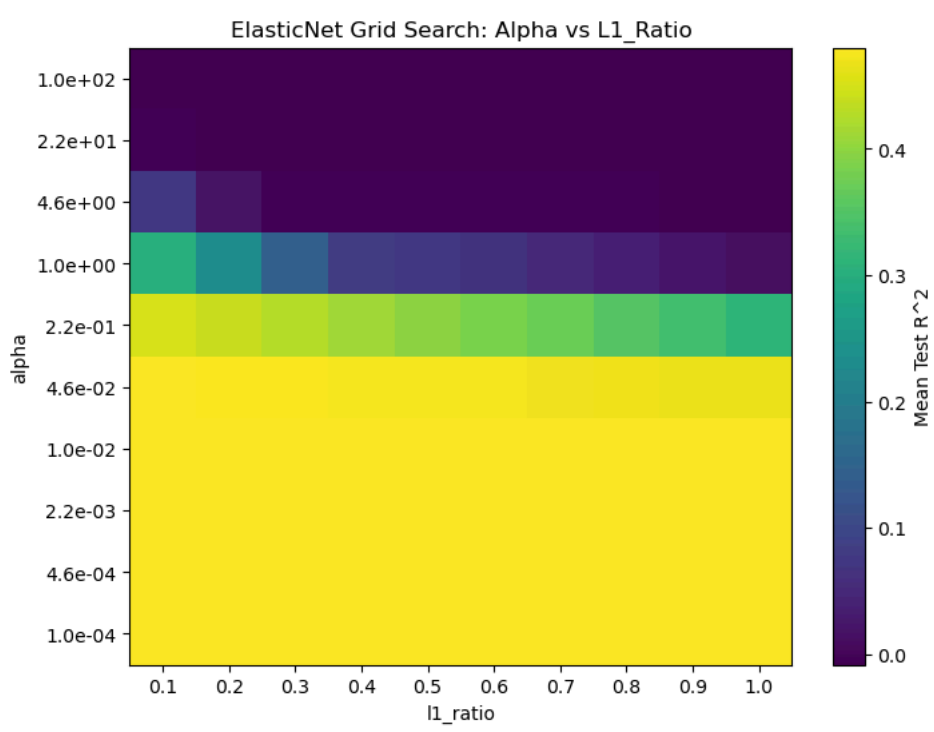
Run a grid search over both parameters to compute cross-validation scores on a 2D grid. Then use the provided function to draw a heatmap and interpret it. What pattern do you see, and how do `alpha` and `l1_ratio` influence each other?

Exercise 2.3 - BALINGIT

ElasticNet blends L1 and L2 regularization. The `l1_ratio` parameter controls how much weight goes to L1 (Lasso) versus L2 (Ridge), so ElasticNet has **two** interacting hyperparameters: `l1_ratio` and `alpha`.

Run a grid search over both parameters to compute cross-validation scores on a 2D grid. Then use the provided function to draw a heatmap and interpret it. What pattern do you see, and how do `alpha` and `l1_ratio` influence each other?

```
from sklearn.datasets import make_regression
if 'X' not in locals():
    X, y = make_regression(n_samples=100, n_features=10, noise=0.1, random_state=42)
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import ElasticNet
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np


# 1. Define the grid of values
param_grid = {
    'alpha': np.logspace(-4, 2, 10),
    'l1_ratio': np.linspace(0.1, 1, 10)  # start from 0.1, not 0
}


# 2. Run the Grid Search
grid = GridSearchCV(ElasticNet(max_iter=10000), param_grid, cv=5, scoring='r2')
grid.fit(X, y)

# 3. Format results for the heatmap
results_df = pd.DataFrame(grid.cv_results_)
pivot = results_df.pivot(index='param_alpha', columns='param_l1_ratio', values='mean_test_score')

# 4. Plot using Matplotlib
fig, ax = plt.subplots(figsize=(8, 6))
im = ax.imshow(pivot, aspect='auto', origin='lower', cmap='viridis')

# Add labels and colorbar
fig.colorbar(im, ax=ax, label='Mean Test R^2')
ax.set_xticks(np.arange(len(pivot.columns)))
ax.set_xticklabels([f"{x:.1f}" for x in pivot.columns])
ax.set_yticks(np.arange(len(pivot.index)))
ax.set_yticklabels([f"{x:.1e}" for x in pivot.index])
ax.set_xlabel('l1_ratio')
ax.set_ylabel('alpha')
ax.set_title('ElasticNet Grid Search: Alpha vs L1_Ratio')

plt.show()

print(f"Best Score: {grid.best_score_:.4f}")
print(f"Best Parameters: {grid.best_params_}")
```




```
Best Score: 0.4801
Best Parameters: {'alpha': np.float64(0.01), 'l1_ratio': np.float64(0.1)}
```

Exercise 3: Coefficient comparison

As a final check, compare the coefficients learned by the optimized linear models.

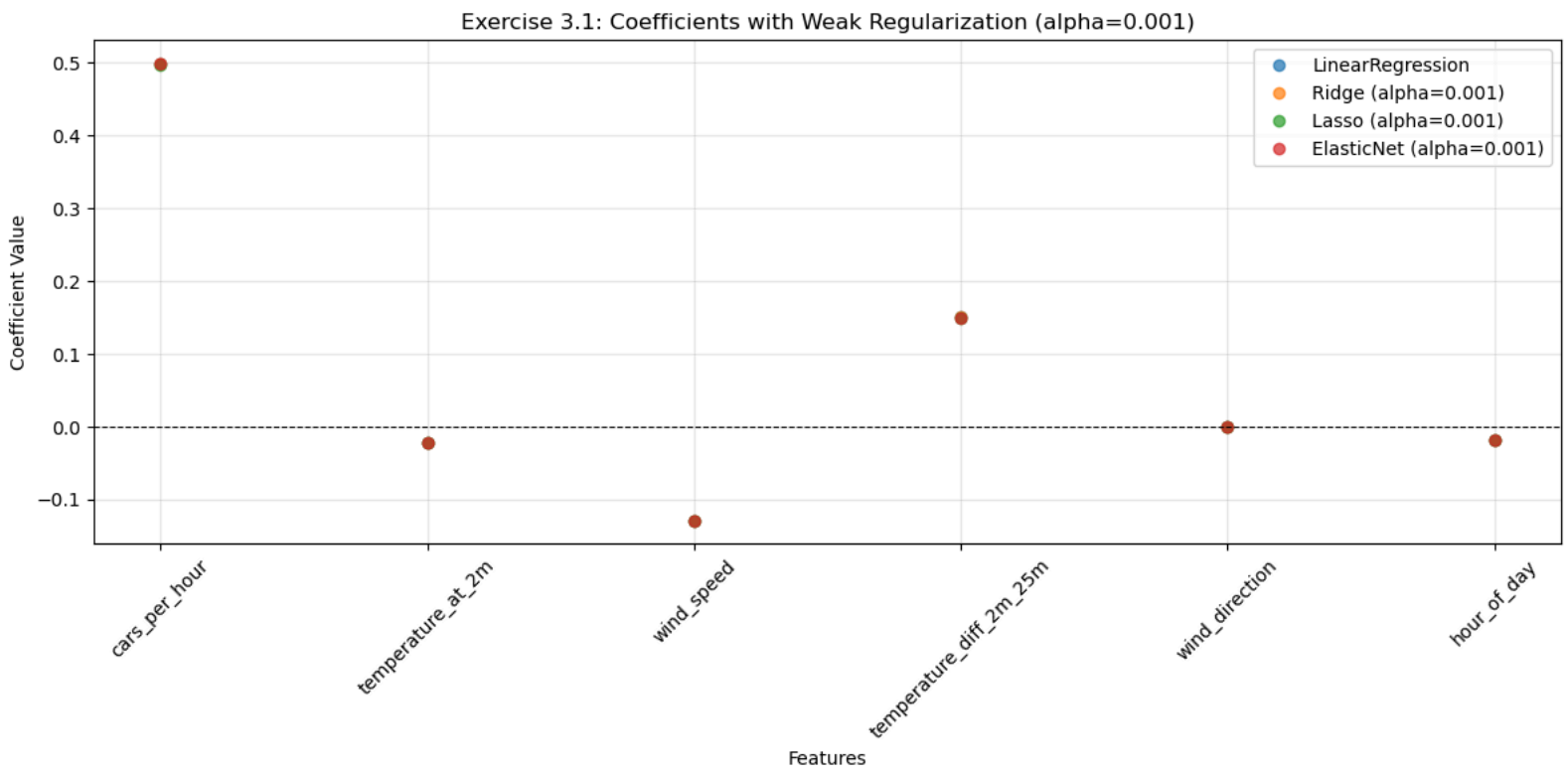
Exercise 3.1 - AQUINO

Make a [scatter plot]() of the coefficients, using different colors for different models. Are the learned weights broadly similar, or do you see clear differences?

For each model, pick an `alpha` that looked reasonable in Exercise 2. If you're unsure, use `alpha=0.001`.

```
models_weak = {
    "LinearRegression": LinearRegression(),
    "Ridge (alpha=0.001)": Ridge(alpha=0.001),
    "Lasso (alpha=0.001)": Lasso(alpha=0.001),
    "ElasticNet (alpha=0.001)": ElasticNet(alpha=0.001, l1_ratio=0.5)
}

coefficients_weak = {}
feature_names = X.columns

for name, model in models_weak.items():
    model.fit(X, y)
    coefficients_weak[name] = model.coef_

plt.figure(figsize=(12, 6))

for name, coefs in coefficients_weak.items():
    plt.plot(feature_names, coefs, marker='o', linestyle='None', label=name, alpha=0.7)
```

```
plt.title("Exercise 3.1: Coefficients with Weak Regularization (alpha=0.001)")
plt.xlabel("Features")
plt.ylabel("Coefficient Value")
plt.axhline(0, color='black', linestyle='--', linewidth=0.8)
plt.legend()
plt.xticks(rotation=45)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

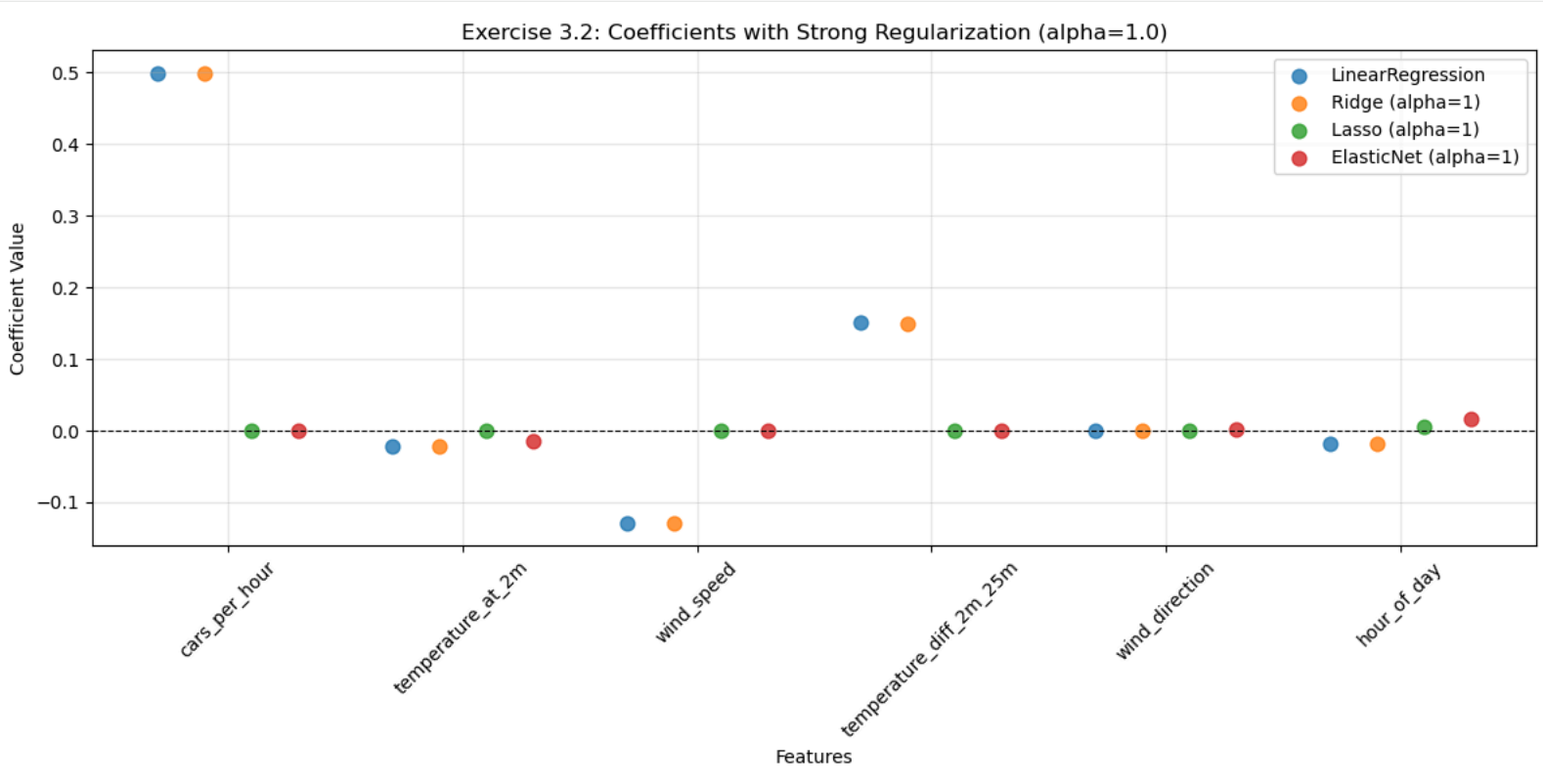


Yes. With an alpha of 0.001, the models are similar. There are no clear differences between the models. The circles of all four models are tightly clustered together.

Exercise 3.2 - AQUINO

Repeat the coefficient plot, but now apply much stronger regularization (e.g., `alpha=1`). What changes? Does this help you connect the earlier performance differences between Ridge and Lasso (Exercise 1.2) to how their coefficients behave?

```
models_strong = {
    "LinearRegression": LinearRegression(),
    "Ridge (alpha=1)": Ridge(alpha=1),
    "Lasso (alpha=1)": Lasso(alpha=1),
    "ElasticNet (alpha=1)": ElasticNet(alpha=1, l1_ratio=0.5)
}

coefficients_strong = {}

for name, model in models_strong.items():
    model.fit(X, y)
    coefficients_strong[name] = model.coef_

plt.figure(figsize=(12, 6))

x = np.arange(len(feature_names))
n_models = len(coefficients_strong)
width = 0.2  # spread between series so points don't overlap

for i, (name, coefs) in enumerate(coefficients_strong.items()):
    offset = (i - (n_models - 1) / 2) * width
    plt.scatter(x + offset, coefs, marker='o', label=name, alpha=0.8, s=60)

plt.title("Exercise 3.2: Coefficients with Strong Regularization (alpha=1.0)")
plt.xlabel("Features")
plt.ylabel("Coefficient Value")
plt.axhline(0, color='black', linestyle='--', linewidth=0.8)
plt.legend()
plt.xticks(x, feature_names, rotation=45)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



At an alpha of 1, the coefficients for Lasso and ElasticNet drop completely to zero. This means that the model effectively deletes important features. This connects to the poor performance from 1.2 as the models discarded the data necessary to predict accurately. With Ridge, it